✨ Module 2

12 October 2026 20:30

Q11. Explain the if-else statement with syntax and example.

Answer:

The **if-else statement** is a decision-making statement in C that executes one block of code when a condition is true and another block when the condition is false.

Syntax:

```c
if(condition)
{
    // statements if condition is true
}
else
{
    // statements if condition is false
}
```

Explanation:

1. The condition is evaluated first.
2. If the condition is **true**, the statements inside the `if` block are executed.
3. If the condition is **false**, the statements inside the `else` block are executed.
4. It is used for making decisions based on a condition.

Example:

```c
#include<stdio.h>

int main()
{
    int num = 10;

    if(num > 0)
        printf("Positive Number");
    else
        printf("Negative Number");

    return 0;
}
```

Output:

```
Positive Number
```

Q12. Explain the working of for, while, and do-while loops in C with their respective syntax structures. Also, compare the primary difference between entry-controlled and exit-controlled loops using a suitable example.

Answer:

A **loop** is used to execute a set of statements repeatedly until a specified condition becomes false.

1. for Loop

Working:

The `for` loop performs initialization, condition checking, and updation in a single statement. It is generally used when the number of iterations is known.

Syntax:

C

```c
for(initialization; condition; updation)
{
    // statements
}
```

Example:

C

```c
for(int i=1; i<=5; i++)
{
    printf("%d ", i);
}
```

Output:

```
1 2 3 4 5
```

2. while Loop

Working:

The `while` loop checks the condition before executing the loop body. If the condition is false initially, the loop does not execute.

Syntax:

```c
while(condition)
{
    // statements
}
```

Example:

```c
int i=1;

while(i<=5)
{
    printf("%d ", i);
    i++;
}
```

Output:

```
1 2 3 4 5
```

3. do-while Loop

Working:

The `do-while` loop executes the loop body first and then checks the condition. Therefore, it executes at least once.

Syntax:

C

```c
do
{
    // statements
}while(condition);
```

Example:

C

```c
int i=1;

do
{
    printf("%d ", i);
    i++;
}while(i<=5);
```

Output:

```
1 2 3 4 5
```

Difference Between Entry-Controlled and Exit-Controlled Loops

Entry-Controlled Loop	Exit-Controlled Loop
Condition is checked before execution.	Condition is checked after execution.
Loop may execute zero times.	Loop executes at least once.
Examples: `for`, `while`	Example: `do-while`
Used when execution depends on initial condition.	Used when one execution is mandatory.

Suitable Example:

Entry-Controlled (`while`)

C

```c
int i=10;

while(i<5)
{
    printf("Hello");
}
```

Output: No output (condition is false initially)

Exit-Controlled (`do-while`)

C

```c
int i=10;

do
{
    printf("Hello");
}while(i<5);
```

Output:

```
Hello
```

Thus, an entry-controlled loop checks the condition before execution, whereas an exit-controlled loop checks the condition after execution.

Q13. Write a C Program to Check Whether a Given Number is Positive, Negative or Zero.

Algorithm:

1. Start the program.
2. Accept a number from the user.
3. Check whether the number is greater than zero.
4. If not, check whether it is less than zero; otherwise it is zero.
5. Display the result and stop.

C Program:

C

```c
#include<stdio.h>

int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if(num > 0)
        printf("Positive Number");
    else if(num < 0)
        printf("Negative Number");
    else
        printf("Zero");

    return 0;
}
```

Sample Output 1:

```
Enter a number: 25
Positive Number
```

Sample Output 2:

```
Enter a number: -8
Negative Number
```

Sample Output 3:

```
Enter a number: 0
Zero
```

Q14. Write a C program that accepts an integer choice and two integer numbers from the user, then uses a switch-case statement to perform addition, subtraction, multiplication, or division based on the choice, handling division by zero appropriately.

Logic

The program accepts two numbers and a choice from the user. Using a `switch-case` statement, it performs the selected arithmetic operation and checks for division by zero before division.

Algorithm

1. Start.
2. Accept two integers and a choice.
3. Use `switch(choice)`.
4. Perform addition, subtraction, multiplication, or division.
5. Check division by zero before division.
6. Display result.
7. Stop.

Program

C

```c
#include<stdio.h>

int main()
{
    int a, b, choice;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    printf("\n1.Addition");
    printf("\n2.Subtraction");
    printf("\n3.Multiplication");
    printf("\n4.Division");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("Sum = %d", a+b);
            break;
```

```c
        case 2:
            printf("Difference = %d", a-b);
            break;

        case 3:
            printf("Product = %d", a*b);
            break;

        case 4:
            if(b==0)
                printf("Division by zero is not possible");
            else
                printf("Quotient = %.2f",(float)a/b);
            break;

        default:
            printf("Invalid Choice");
    }

    return 0;
}
```

Sample Output

```
Enter two numbers: 20 10
Enter your choice: 1


Sum = 30
```

Q15. Write a C program to print a square and a right-angled star pattern using nested loops.

Logic

Nested loops are used where one loop controls rows and another loop controls columns.

A) Square Pattern

Pattern

```
* * * *
* * * *
* * * *
* * * *
```

Program

C

```c
#include<stdio.h>

int main()
{
    int i,j;

    for(i=1;i<=4;i++)
    {
        for(j=1;j<=4;j++)
        {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}
```

B) Right-Angled Triangle Pattern

Pattern

```
*
* *
* * *
* * * *
```

Program

C

```c
#include<stdio.h>

int main()
{
    int i,j;

    for(i=1;i<=4;i++)
    {
        for(j=1;j<=i;j++)
        {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}
```

Q16. Write a C program to print a pyramid pattern using nested loops.

Logic

A pyramid consists of spaces followed by stars. Nested loops are used to print spaces and stars separately.

Pattern

```
   *
  ***
 *****
*******
```

Algorithm

1. Start.
2. Use outer loop for rows.
3. Print spaces.
4. Print stars.
5. Move to next line.
6. Stop.

Program

C

```c
#include<stdio.h>

int main()
{
    int i,j;

    for(i=1;i<=4;i++)
    {
        for(j=1;j<=4-i;j++)
        {
            printf(" ");
        }

        for(j=1;j<=2*i-1;j++)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Sample Output

```
   *
  ***
 *****
*******
```